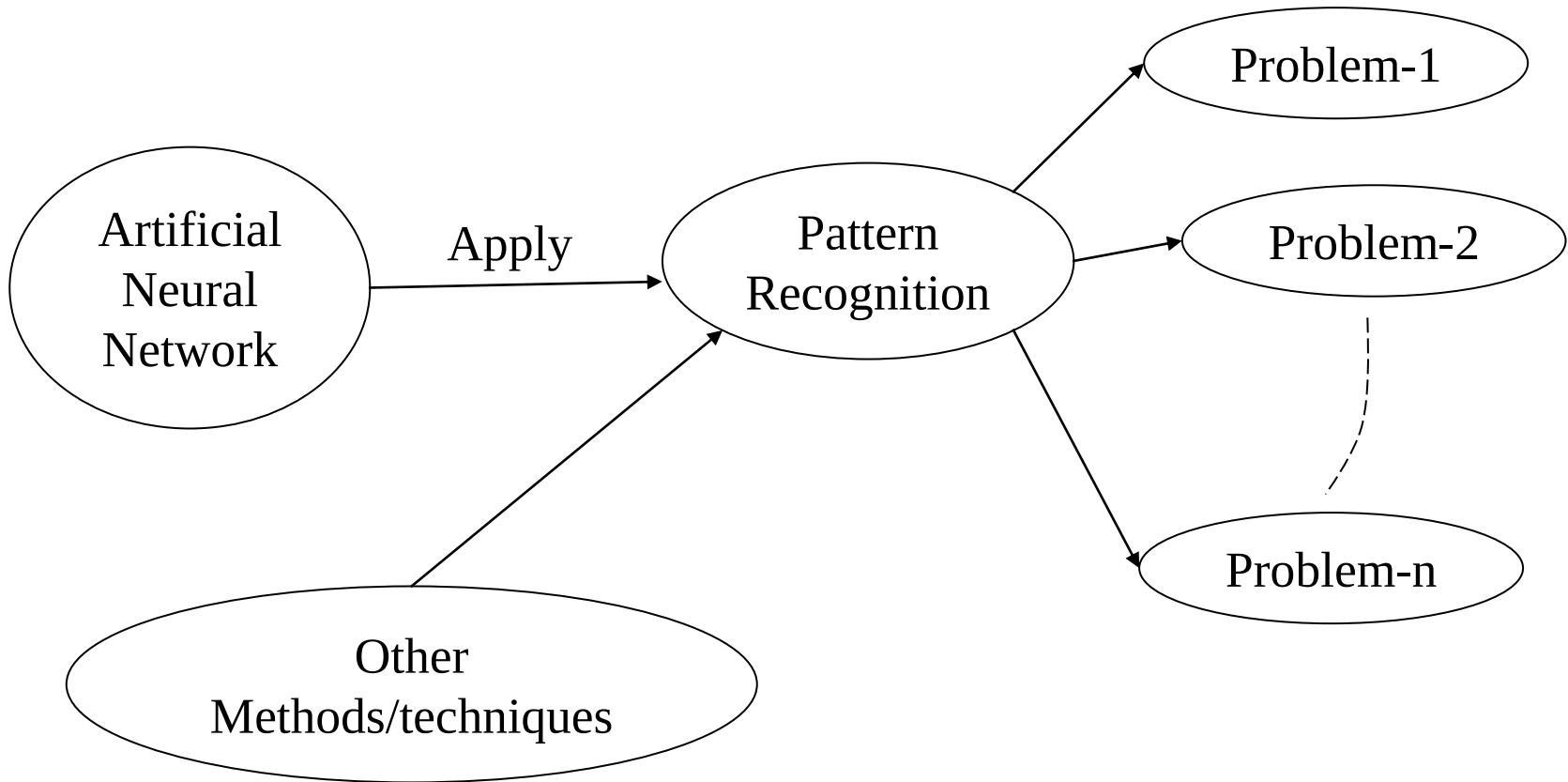


# Neural Networks and Fuzzy Systems

## Pattern Recognition

Sakurah Ismail  
Lecturer  
Dept. of CSE  
NBIU

# Introduction

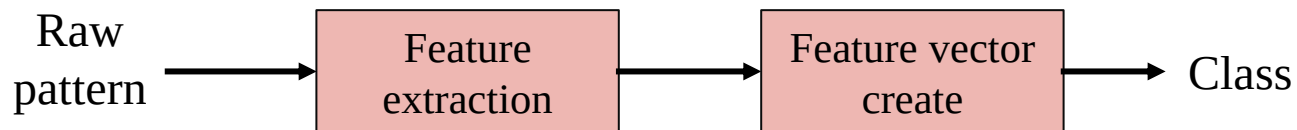


# Pattern Recognition In Perspective

- ◆ Patterns → alphabetic characters, text processing, handwritten text, speech, images, stock market trends, etc.
- ◆ One of the **simpler pattern recognition** tasks → Classification
  - Simplest classifier – **template matching technique** –
    - each letter is read into a fixed size frame and the frame is compared to a template of all the possible characters but cannot apply to all sizes.
  - **Problem** → change typeface of the text/ new font / handeritten text
- ◆ Complex domain → image processing, speech and stock market tends

# Pattern Recognition - A Definition

- ◆ A pattern recognition system can be considered as a **two-stage device**
  - **Feature extraction**
    - a measurement taken on the input pattern
    - Example : to distinguish 'E' and 'F' – we need to compare the number of vertical and horizontal strokes.
  - **Classification**
    - Map these input features onto a classification state
      - Given the input features, the classifier must decide which type of class category they match most closely
        - i.e. – **distance metrics** and **probability theory**.

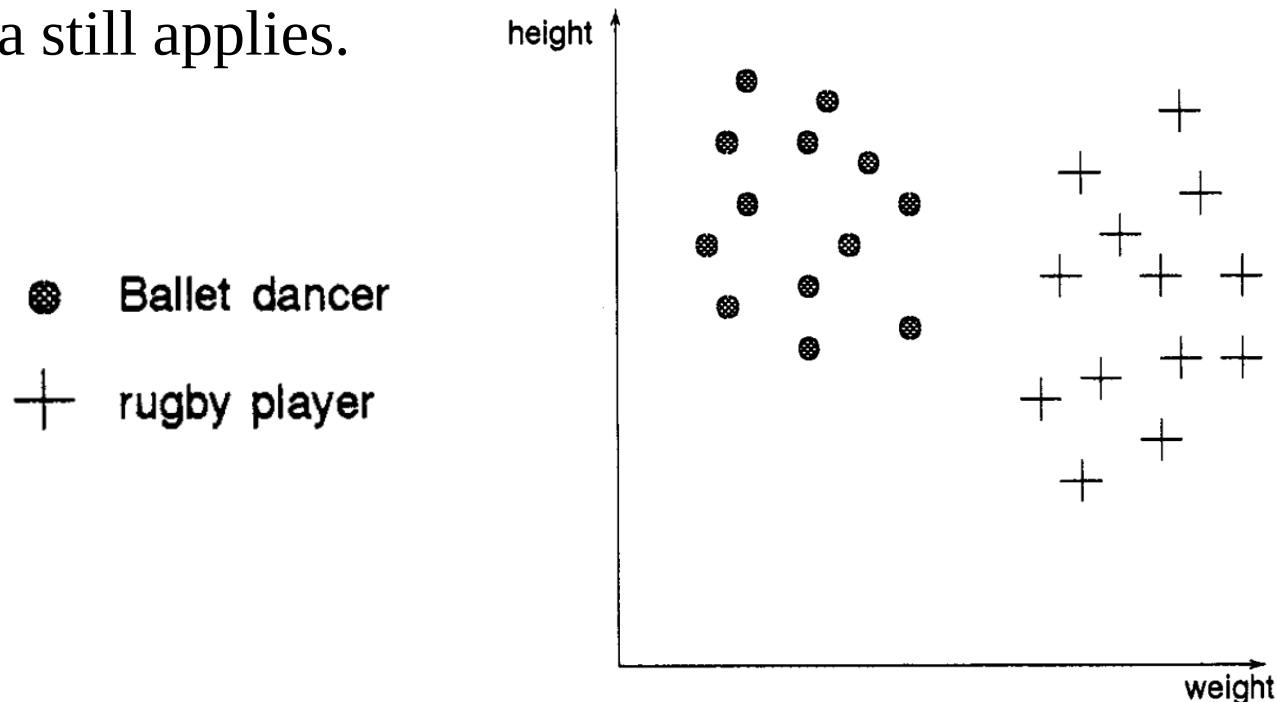


# Feature Vectors and Feature Space

- ◆ If we make **n measurements** on our input pattern, each of which is a unique feature, then we can use algebraic notation to **create a set of these features** and call it a **feature vector**.
- ◆ The dimensionality of the vector – the number of elements in it – creates an **n dimensional feature space**.
- ◆ When trying to classify inputs (like images or people), we measure certain features—like height and weight.
- ◆ These features form a feature vector, and if we use n features, we get an n-dimensional feature space.

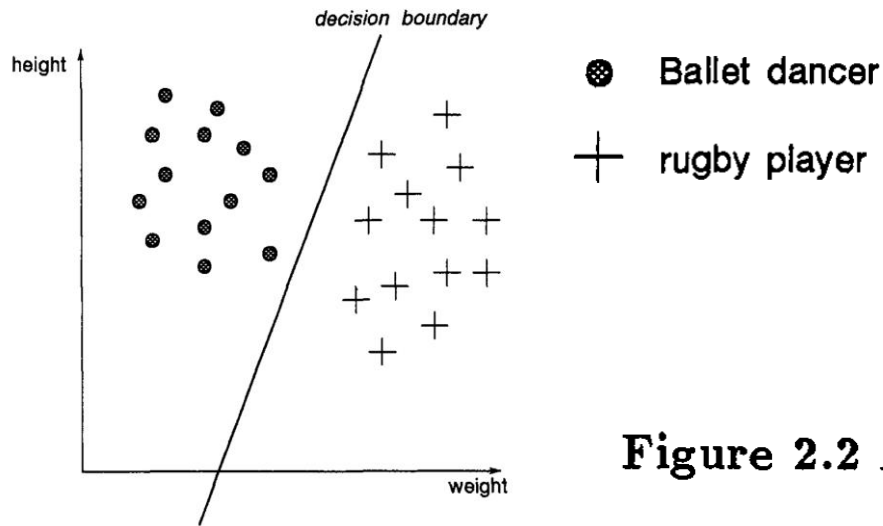
# Feature Vectors and Feature Space

- ◆ For example, to tell ballet dancers apart from rugby players, we could use just two features: height and weight.
- ◆ Plotting these on a 2D graph helps us see how they differ.
- ◆ With more than three features, it becomes harder to visualize, but the same idea still applies.



**Figure 2.1** A two-dimensional Euclidean feature space.

# Discriminant Functions



**Figure 2.2** A linear classification decision boundary.

- ◆ Fig. 2.2 – rugby-player/ ballet-dancer classification problem – samples form **two distinct cluster** – we could **define boundary/line** to separate them – a **decision boundary** – **discriminating function**.
- ◆ **A function** that discriminate one class from another class.

# Discriminant Functions

- ◆ The simplest function that separate the two clusters is a **straight line**.
- ◆ It represents a very widely used category of classifiers known as **linear classifiers**.

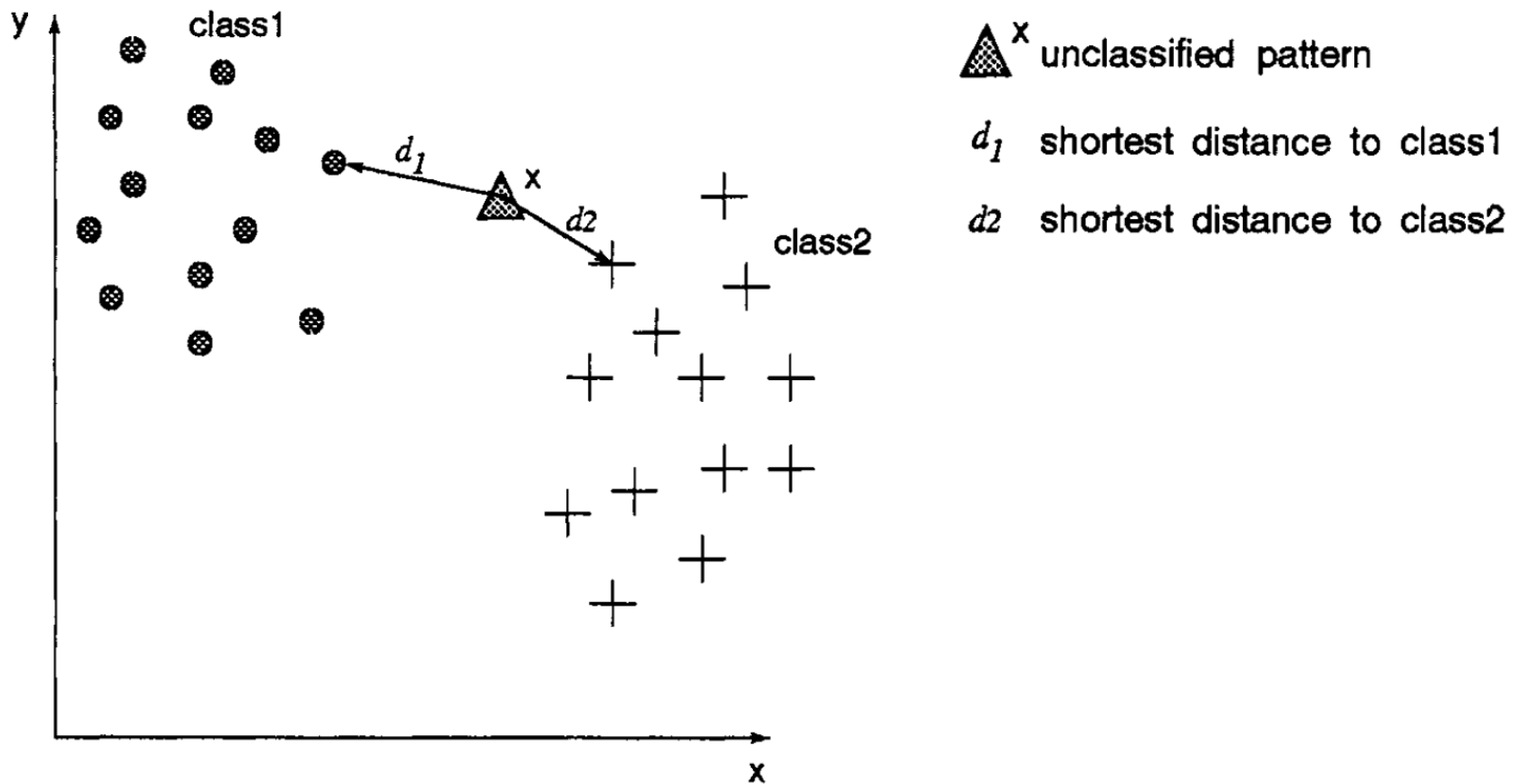
# Classification Techniques

- ◆ Pattern classification techniques :
  - numeric and non-numeric.
- ◆ Numeric technique –
  - **deterministic** and **statistical** measures – made on the geometric pattern space.
- ◆ Non-numeric techniques –
  - based on **symbolic processing** – i.e. – **fuzzy set**.
- ◆ **Our concern** - the numeric techniques –
  - followed the same by **neural computing** since symbolic data manipulation is based on traditional AI approach.

# Classification Techniques

- ◆ To implement **discriminant function analysis**:
  - **Nearest neighbor classification**: Distance metrics, Euclidean, City Block.
  - **Linear classifier**
  - **Statistical techniques**: Bayesian classifier
  - **Neural Networks**

# Nearest neighbor classification

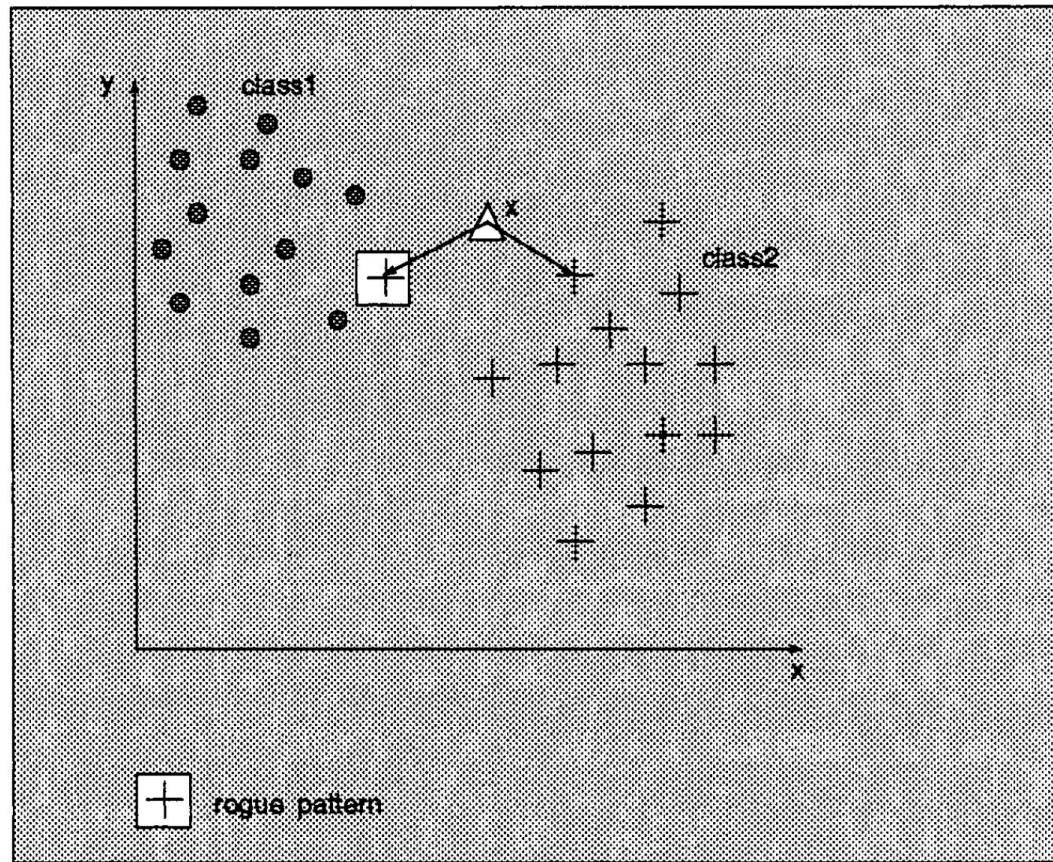


**Figure 2.3** Classification by comparison to the “nearest neighbour”.

# Nearest neighbor classification

- ◆ We wish to decide to which of the two the **unclassified pattern**, **X**, belongs.
- ◆ We can define **discriminating function** as:
  - $f(X) = \text{closest}(\text{class1}) - \text{closest}(\text{class2})$
  - $f(X) > 0 \rightarrow \text{Belongs to class2}$
  - $f(X) < 0 \rightarrow \text{Belongs to class1}$

# Nearest neighbor classification



**Figure 2.4:** Nearest neighbor method can give wrong results if a bad (rogue) sample is chosen.

# Nearest neighbor classification

- ◆ Sometimes a sample (like in Figure 2.4) belongs to one group, but it is physically closer to another group.
- ◆ This can confuse the system and lead to wrong classification.
- ◆ If we compare a new input to such a bad or unusual sample, we might end up classifying it incorrectly.
- ◆ To fix this problem, we don't rely on just one sample. Instead, we look at the average of several nearby samples.
- ◆ This is called "K-nearest neighbor" classification, where K is the number of nearby samples we check.

# Distance Metrics

- ◆ Nearest Neighbor pose problem of finding the distance.
- ◆ Methods of **measurement based on distance**:
  - Hamming distance
  - Euclidean distance
  - City block distance

# Distance Metrics

## ◆ Hamming distance

- For two vector

$$X = (x_1, x_2, \dots \dots \dots)$$

$$Y = (y_1, y_2, \dots \dots \dots)$$

- The Hamming distance is found by evaluating the difference between each component of one vector with the corresponding component of the other, and summing these differences to provide an absolute value for the variation between the two vectors.
- The measure is defined by:

$$H = \sum (|x_i - y_i|)$$

# Distance Metrics

## ◆ Hamming distance

- The Hamming distance is often used to **compare binary vectors**.
- In actual fact the Hamming distance measure for binary data can be **performed simply** by **the exclusive-OR function** since

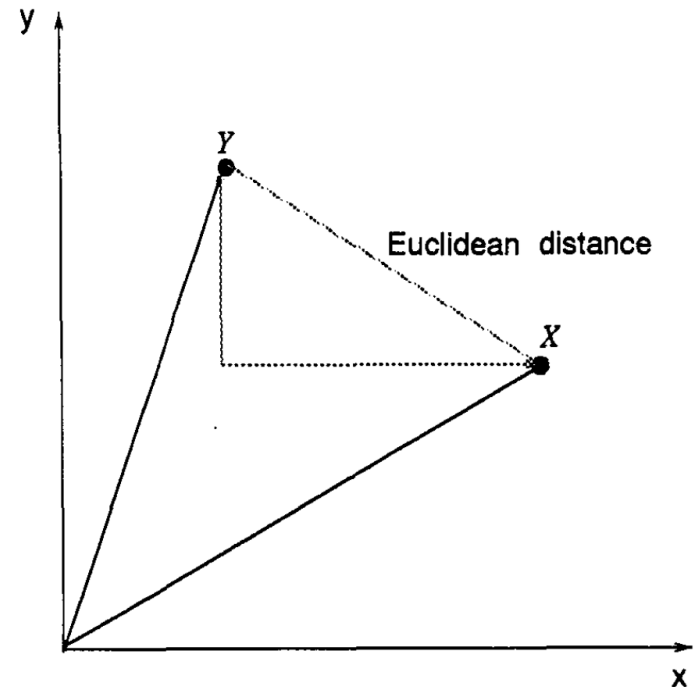
$$|x_i - y_i| \text{ is equivalent to } x_i \text{ XOR } y_i$$

# Distance Metrics

- ◆ Euclidean distance
  - Two vectors ( X and Y)
  - The Euclidean distance which is defined by:

$$d(X, Y)_{euc} = \sqrt{\sum_{i=1}^n (X_i - Y_i)^2}$$

- where **n** is the dimensionality of the vector.



**Figure 2.5** The Euclidean distance measure.

# Distance Metrics

## ◆ Euclidean distance

- For the **two-dimensional** example we have drawn, this gives us:

$$d(X, Y)_{euc} = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2}$$

- A special case is given **for binary vectors** where the metric is then equivalent to the square root of the Hamming distance

$$d(X, Y) = \sqrt{\left(\sum_{i=1}^n (x_i - y_i)\right)^2} = \sqrt{\left(\sum_{i=1}^n (x_i \text{ XOR } y_i)\right)^2}$$

- The metric is used in a **neural network learning algorithm**

# Distance Metrics

## ◆ City block distance (Manhattan)

- A **simplified version of the Euclidean distance** measure is the city block measure.
- This method performs the Euclidean measure **without calculating the squared or square root functions**. Thus

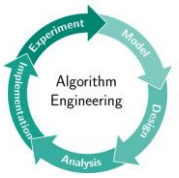
$$D_{cb} = \sum_n |X_j - Y_j|$$

- It is much **faster** to compute than the Euclidean



# Problem 1: Euclidean Distance

---



Given two feature vectors

$$X=(2,3), Y=(5,7)$$

Calculate the **Euclidean distance** between them.

The Euclidean distance is

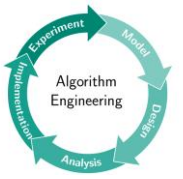
$$d(X,Y)=\sum n(x_i-y_i)^2$$

So,

$$\begin{aligned} d(X,Y) &= (2-5)^2 + (3-7)^2 \\ &= (-3)^2 + (-4)^2 \\ &= 9 + 16 \\ &= 25 \\ &= 5 \end{aligned}$$

# Problem 2: City Block Distance

---



Given

$$X=(2,3), Y=(5,7)$$

Calculate the **City Block distance**.

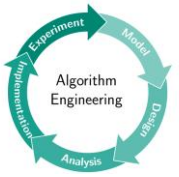
$$d(X,Y)=\sum_{i=1}^n |x_i - y_i|$$

Therefore,

$$d(X,Y)=|2-5|+|3-7| = 3+4 \quad d(X,Y)=7$$

# Problem -3

---



Consider two known feature vectors  $A=(2,3)$  belonging to Class 1 and  $B=(7,7)$  belonging to Class 2. An unknown pattern is represented by  $X=(4,4)$ . Classify  $X$  using the nearest-neighbor method.

# Summary

- ◆ **Pattern recognition** → feature extraction and classification.
- ◆ **Features** are pattern measurements used for comparison.
- ◆ **Discriminant functions** partition up feature space.
- ◆ A number of **different distance metrics** are used.